

ReelWalk: short things to say

One idea per line. Say the English until it's automatic, then the Japanese. If you get stuck in the interview, reach for the picture on each group's right: it brings the lines back. Furigana only on words above N2.

What it is どんなアプリ？

Phone in, Reel out.

1. ReelWalk makes Instagram Reels from property photos, on a phone.

ReelWalkは、物件の^{ぶっけん}写真から、スマホでリール動画を作るアプリです。

2. Its special feature is a floor-plan marker that moves with the video.

特徴は、動画と一緒に動く、^{まどず}間取り図のマーカーです。

3. Restaurants can make Reels too, from their menu and their own clips.

レストランも、メニューと自分の動画からリールを作れます。

4. A finished video can be shared with a link that works anywhere.

できた動画は、^{きょうゆう}共有リンクで、どこでも見られます。

5. It's live today at reelwalking.com.

今は、reelwalking.comで本当に動いています。

How it works どう動く？

Web is the waiter, the queue is the order ticket, the worker is the kitchen.

6. The web app is the waiter, the queue is the order ticket, the worker is the kitchen.

Webはウェイター、キューは^{ちゅうもんひょう}注文票、ワーカーはキッチンです。

7. The web app never makes videos. It only takes the order.

Webアプリは、動画を作りません。注文を受けるだけです。

8. The database is the only source of truth. The queue only carries an ID.

正しい情報はデータベースだけ。キューにはIDしか入れません。

9. Videos upload straight from the phone to S3.

動画は、スマホからS3へ直接アップロードします。

10. While editing, the phone plays a light 720p copy; the export uses the original.

編集中は軽い720pのコピーを再生して、書き出しは元の動画を使います。

11. Preview and export use the same component, so what you see is what you get.

プレビューと書き出しは同じ部品なので、見たままの動画になります。

12. I designed it assuming a message can arrive twice.

メッセージは二回届くかもしれない、と考えて作りました。

13. If it runs twice, there's still one result. That's idempotency.

二回動いても、結果は一つです。これが冪等性^{べきとうせい}です。

14. If a worker dies, another one takes over within about two minutes.

ワーカーが止まっても、二分ぐらいで別のワーカーが引き継ぎ^{ひきつぎ}ます。

15. After three failures the job is marked failed, and the user can retry.

三回失敗したら、ジョブは失敗になって、ユーザーがやり直せます。

16. The job and its message are saved in the same transaction (an outbox).

ジョブとメッセージは、同じトランザクションで保存します。

17. In a load test, I made 90 videos and killed a worker midway. All 90 succeeded.

テストで90本作って、途中でワーカーを止めても、全部成功しました。

AWS and money AWSとお金**18. First I ran it on EKS for a day, then I looked at the bill.**

最初はEKSで一日動かして、それから請求額^{せいきゅうがく}を見ました。

19. \$250 a month for something mostly idle. That didn't fit.

ほとんど使わないのに、月250ドル。それは合いませんでした。

20. Now it's App Runner plus Fargate only when needed: about \$10 to \$15 a month.

今はApp Runnerと、必要な時だけ動くFargateで、月10ドルから15ドルくらいです。

21. When nobody is exporting, no render server is running at all.

書き出しがない時は、サーバーは一台も動いていません。

22. The trade-off: the first export after a quiet spell takes a minute or two to start.

その代わり、最初の書き出しは、始まるまで一、二分かかります。

23. Kubernetes isn't gone: it still runs locally on kind, with Argo CD.

Kubernetesは、今もローカルのkindで、Argo CDと一緒に動いています。

24. Every change is tested and scanned before it ships.

変更は全部、テストとスキャンをしてから出します。

25. All the AWS resources are managed in Terraform.

AWSのリソースは、全部Terraformで管理しています。

26. There are no access keys in production, only roles.

本番にアクセスキーはありません。ロールだけです。

27. Each vulnerability found is either fixed, or accepted with a written reason.

見つかった脆弱性^{ぜいじゃくせい}は、直すか、理由を書いて受け入れます。

28. Rules are tested with Vitest, and user flows with Playwright, right up to a real export.

ルールはVitestで、画面の流れはPlaywrightで、本物の書き出しまで確かめます。

Honest and forward-looking 正直に、前向きに*Say what's done, say what's next.*

29. I keep what's done and what's not done clearly separate.

できたことと、まだのことを、はっきり分けて話します。

30. Direct posting to Instagram is built, but it's still in Meta's development mode.

インスタへの直接投稿は作りましたが、まだMetaの開発モードです。

31. I used AI tools too, but every design decision is written down.

AIツールも使いましたが、設計の判断は全部、文書に残しています。

32. My biggest lesson: measure first, then choose.

一番の学びは、測ってから選ぶことです。

33. Next, I want to make that first export start faster.

次は、最初の書き出しを、もっと速くしたいです。